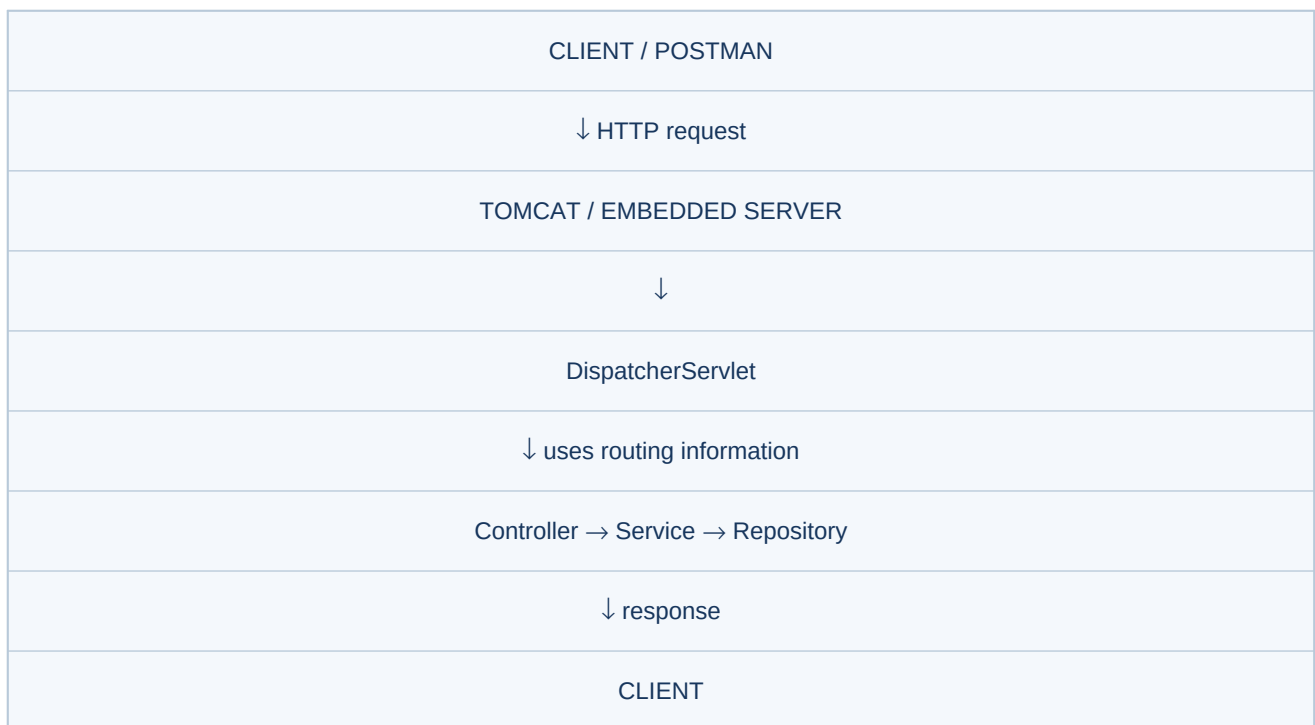


SPRING MVC

Internal Architecture & Manual Configuration

Simple, topic-wise lecture notes with clear diagrams and Java examples



Main goal: understand what happens between an HTTP request and your controller method—and why Spring Boot removes much of the manual setup.

1. What Is Spring MVC?

Spring MVC is Spring Framework's web framework. It helps build web applications by connecting incoming HTTP requests to Java methods.

- **REST APIs:** return data, commonly JSON, to clients such as Postman or a frontend.
- **Web applications:** return a page such as HTML or JSP.
- Spring MVC provides annotations and components such as `@Controller`, `@RestController`, `@GetMapping`, `@PostMapping`, and `@RequestMapping`.
- The module is commonly referred to as Spring MVC; its module name is Spring Web MVC.

Spring MVC
■■■ REST API → JSON response
■■■ Web page → HTML / JSP response

Remember

Spring MVC is the web framework. Spring Boot makes setting up and running a Spring MVC application easier through auto-configuration and sensible defaults.

2. Spring MVC vs Spring Boot

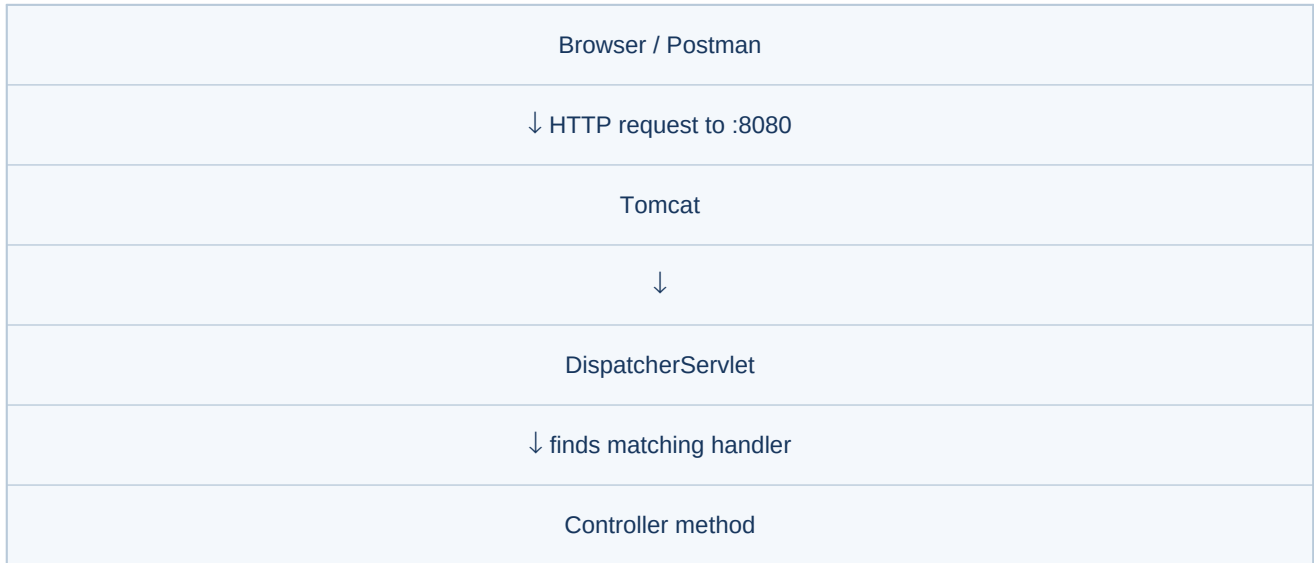
Spring MVC
Web framework
Controllers, mappings, request/response handling
Manual configuration may be required

Spring Boot
Convenience / auto-configuration layer
Configures common web infrastructure
Lets you focus sooner on business logic

- In a typical Spring Boot web project, the starter dependency brings in the common web dependencies.
- In a manually configured Spring MVC project, you may need to configure the server, DispatcherServlet, component scanning, MVC support, and other infrastructure yourself.
- Spring Boot does not replace Spring MVC; it commonly configures and runs it for you.

3. The Missing Link: Who Calls the Controller?

A controller does not directly listen to port 8080. A server accepts the HTTP request, and Spring MVC routes it to the appropriate controller method.



Example

```
GET http://localhost:8080/hello
```

```
@GetMapping("/hello")
public String hello() {
    return "Hello";
}
```

The question this lecture answers is: how does a request for /hello reach the correct method?

4. Traditional Servlet Architecture

Before Spring MVC's controller-based programming model, developers often created multiple servlets and mapped URLs to them.



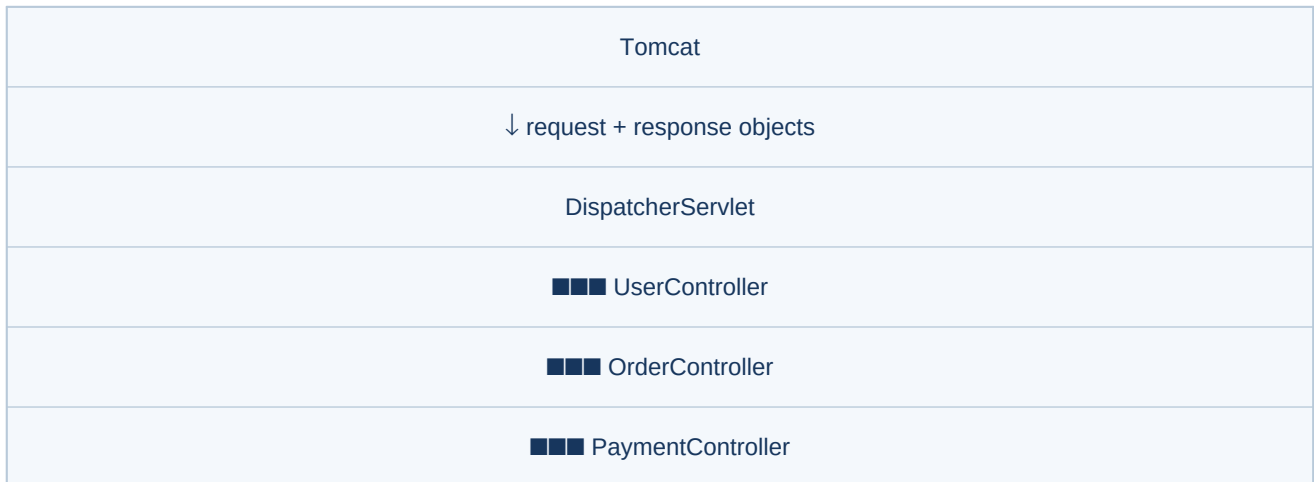
- Tomcat receives and parses the HTTP request.
- It provides request and response objects to the selected servlet.
- The servlet reads request data, runs logic, and writes the response.
- Different servlets may repeat similar infrastructure tasks.

Repeated work in individual servlets

- Read request parameters and body.
- Convert JSON to Java objects and Java objects to JSON.
- Set response status and body.
- Handle errors and exceptions.

5. Why Use a Central DispatcherServlet?

Spring MVC introduces one central servlet—the DispatcherServlet—as the front controller. It receives incoming requests and coordinates routing to controller methods.



- The DispatcherServlet is the central entry point for Spring MVC requests.
- Application developers usually write controller methods instead of creating a servlet for every resource.
- Common request-processing work is handled by Spring MVC infrastructure components.
- The controller can focus on application logic and delegate to service/repository layers.

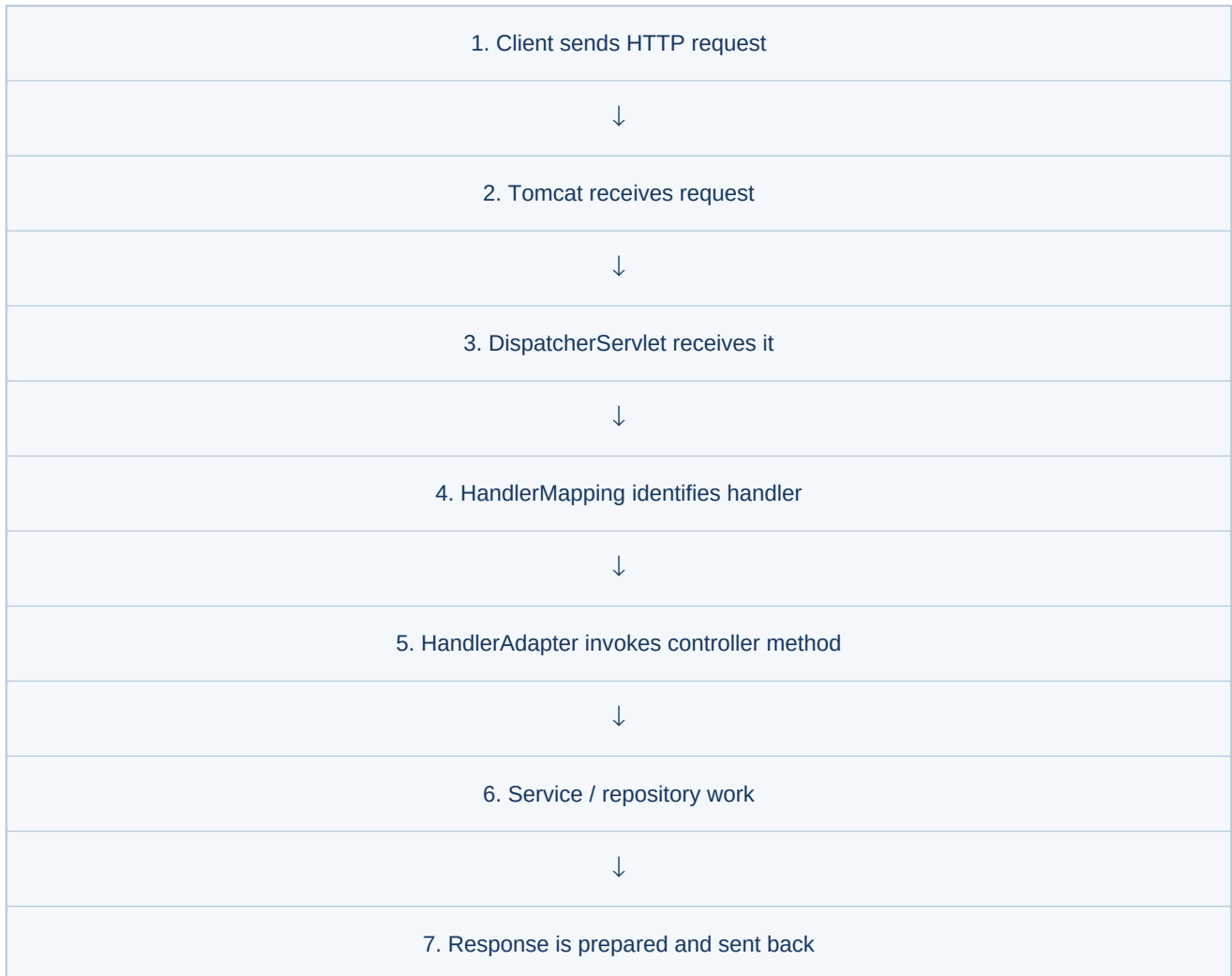
6. What Are HttpServletRequest and HttpServletResponse?

Tomcat creates or provides the servlet request and response objects and passes them into the servlet container pipeline.



- Request object: represents information sent by the client.
- Response object: represents the response that will be sent back.
- In Spring MVC, you often do not manipulate these objects directly because annotations and message converters handle common cases.

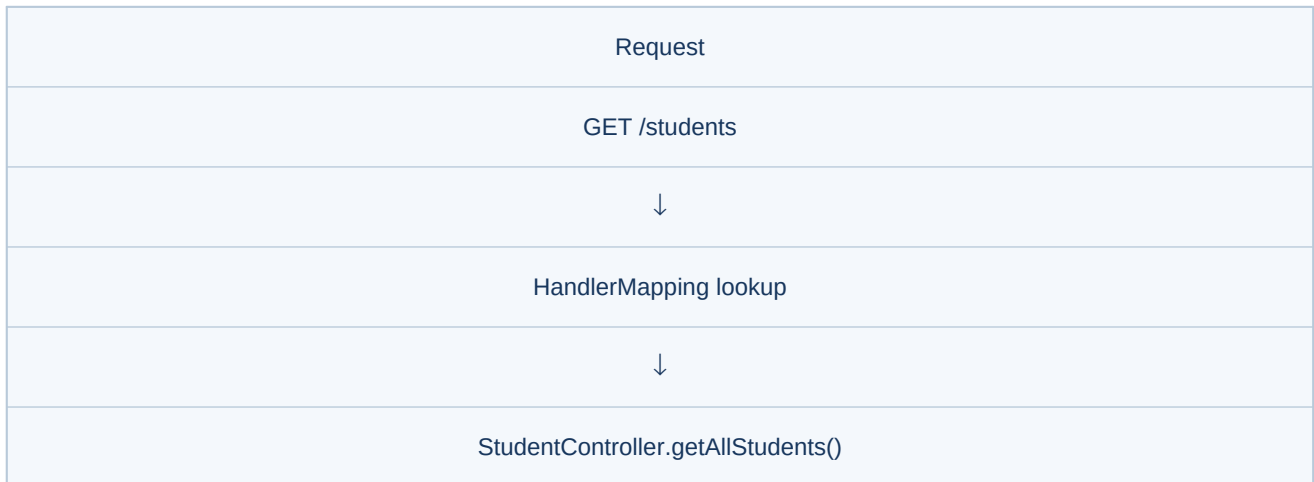
7. DispatcherServlet Request Flow



- This is a simplified mental model; Spring MVC has more internal components.
- HandlerMapping identifies which handler matches the request.
- A HandlerAdapter is used internally to invoke the selected handler.
- For REST responses, an HTTP message converter commonly serializes Java objects as JSON.

8. HandlerMapping — The Route Lookup

HandlerMapping helps Spring MVC determine which controller method matches an incoming request, based on details such as the URL path and HTTP method.



Example mapping table

HTTP method	Path	Handler method
GET	/students	getAllStudents()
POST	/students	createStudent()
GET	/students/{id}	getStudent(id)

How is the mapping information discovered?

During application initialization, Spring's infrastructure discovers controller beans and their mapping annotations, then builds routing information. The DispatcherServlet uses this information to process requests.

9. IoC Container and Controller Beans

The Spring IoC container creates and manages beans. Controllers, services, and repositories can be Spring-managed beans when registered through annotations or configuration.

Spring IoC Container
■■■ StudentController bean
■■■ StudentService bean
■■■ StudentRepository bean

- `@RestController` and `@Controller` mark controller components.
- `@Service` marks a service component.
- `@Repository` marks a repository component.
- `@ComponentScan` tells Spring which package area to scan for components.
- Constructor injection lets Spring provide a dependency when creating a bean.

10. Three Common Ways to Send Data

1. Request body
JSON data
POST /students

2. Path variable
Value is part of the path
GET /students/1

3. Query parameter
Key-value after ?
GET /students?id=1

Quick comparison

- **Request body:** commonly used to send a larger object, such as student details when creating a record.
- **Path variable:** commonly identifies a specific resource, such as student ID 1.
- **Query parameter:** commonly filters or modifies a query, such as searching by name or ID.

11. JSON Request Body → Java Object

For a REST endpoint, Spring MVC can use an HTTP message converter—commonly backed by Jackson—to deserialize JSON into a Java object. The lecture's simplified explanation attributes this work to `DispatcherServlet`; internally, dedicated message-conversion components do the conversion.

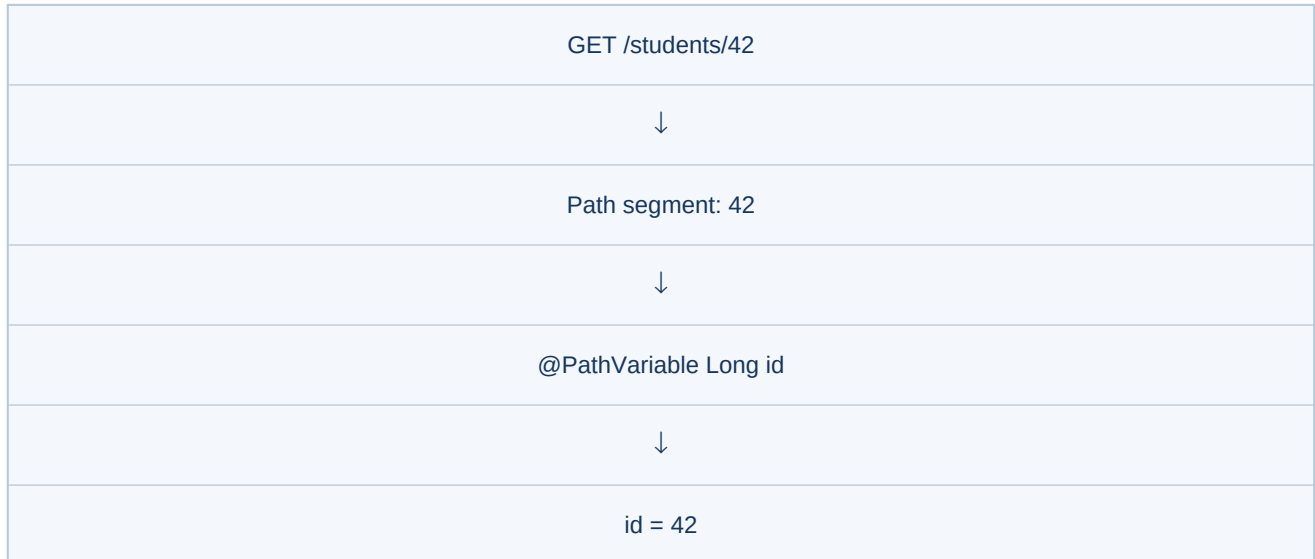
```
// Example JSON body
{
  "id": 1,
  "name": "Aditya",
  "email": "aditya@example.com"
}

// Controller parameter
@PostMapping("/students")
public ResponseEntity<Student> create(
    @RequestBody Student student) {
    // student contains the mapped fields
    return ResponseEntity.ok(student);
}
```

- `@RequestBody` tells Spring to read the request body and bind it to the method parameter.
- JSON field names generally need to match or be configured to map to the Java object's properties.
- The controller does not normally write the JSON parsing code itself.

12. Path Variables

```
@GetMapping("/students/{id}")
public ResponseEntity<Student> getStudent(
    @PathVariable Long id) {
    // use id to find a student
}
```



Spring MVC resolves the path value and supplies it to the controller parameter. Handler method argument resolution is part of the internal mechanism.

13. Query Parameters

```
// Request: GET /students?id=42
```

```
@GetMapping("/search")
public String search(@RequestParam Long id) {
    return "Searching for " + id;
}
```

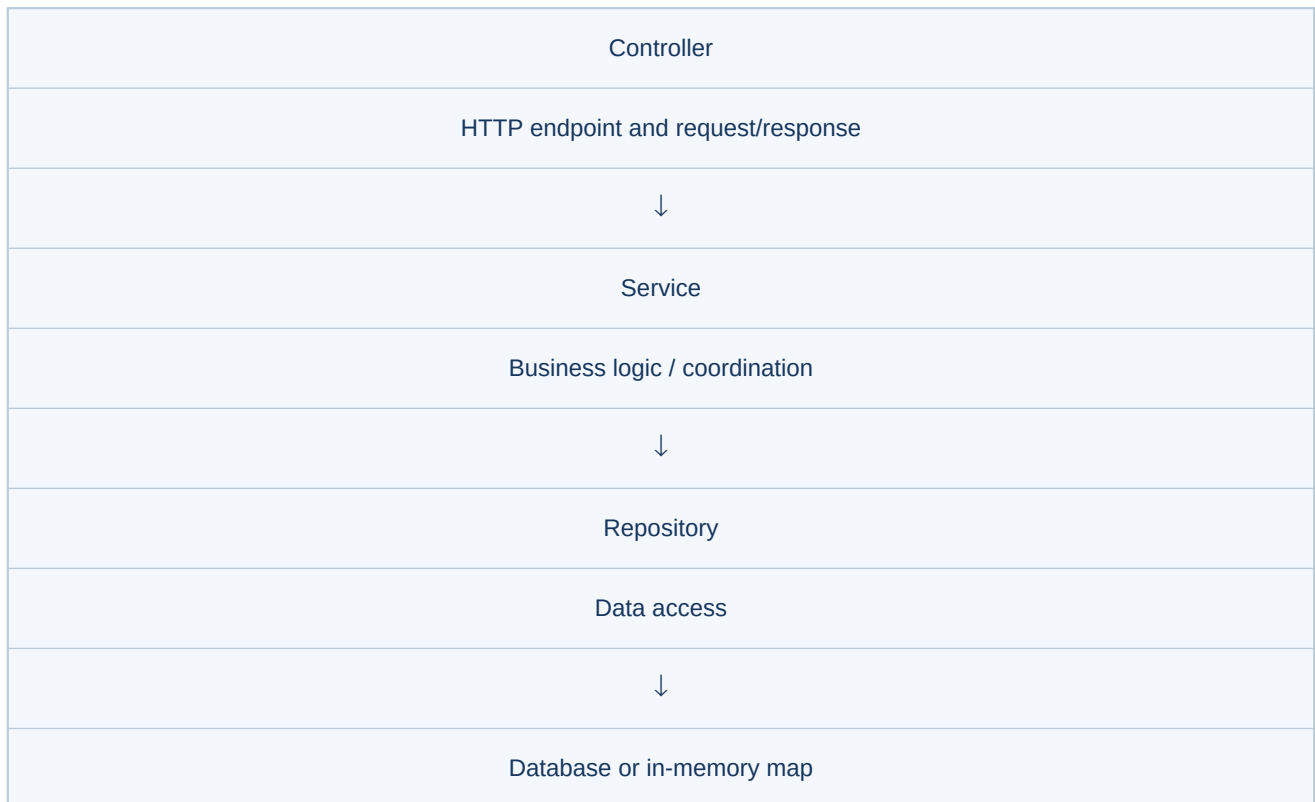
- Query parameters appear after a question mark in the URL.
- Multiple parameters are separated by ampersands, for example `/students?name=Adi&active=true`.
- Servlet request APIs expose query parameters; Spring MVC can bind them to method arguments with `@RequestParam`.

14. Response Flow: Controller Back to Client



- A controller may return a Java object or a `ResponseEntity`.
- For a REST controller, Spring MVC commonly serializes the returned object to JSON.
- The response includes a status code and, when applicable, headers and a body.
- Tomcat sends the resulting HTTP response to the client.

15. Controller → Service → Repository



- Controller: handles the web request and selects the application operation.
- Service: holds business logic and coordinates work.
- Repository: reads and writes data.
- The lecture uses a HashMap to mimic storage so the focus stays on MVC configuration rather than database/JPA setup.

16. Embedded Tomcat vs External Tomcat

External Tomcat
Build WAR file
↓ deploy
Separately running Tomcat

Embedded Tomcat
Tomcat dependency inside app
↓ run application
App starts its own server

- External deployment commonly involves packaging a WAR and deploying it to an already-running Tomcat.
- Embedded Tomcat runs as part of the application process, often with a runnable JAR.
- The HTTP request flow through Tomcat and DispatcherServlet is conceptually similar in both cases.
- The lecture builds a manual Spring MVC app using embedded Tomcat.

17. Manual Project Dependencies

The lecture's manual Maven project adds dependencies individually because it is not relying on Spring Boot's starter and auto-configuration.

- **Spring Web MVC:** provides MVC infrastructure, controllers, mappings, and related support.
- **Embedded Tomcat:** provides the servlet container/server.
- **Jackson databind:** supports converting between JSON and Java objects.
- The exact versions shown in a lecture are examples, not a recommendation to always use those versions. Choose maintained, compatible versions for a real project.

Manual MVC dependencies
Spring Web MVC + Embedded Tomcat + Jackson
↓
More configuration is written by the developer

18. Student Model / POJO

```
package in.dotsquares.entity;

public class Student {
    private Long id;
    private String name;
    private String email;

    // getters and setters
}
```

- This is a simple Java object used to represent student data.
- The lecture uses fields for ID, name, and email.
- It is not automatically a JPA entity just because the package is named `entity`; no database mapping is being configured in this demo.
- Jackson can bind matching JSON properties to the object.

19. StudentRepository Using a HashMap

```
@Repository
public class StudentRepository {
    private final Map<Long, Student> studentDb = new HashMap<>();

    public Student save(Student student) {
        studentDb.put(student.getId(), student);
        return student;
    }

    public Student findById(Long id) {
        return studentDb.get(id);
    }

    public List<Student> findAll() {
        return new ArrayList<>(studentDb.values());
    }
}
```

- The map uses the student ID as the key and the Student object as the value.
- `save()` stores a student, `findById()` retrieves one, and `findAll()` returns all stored values.
- This is in-memory demo storage—not a durable database. Data is lost when the application restarts.
- For a real multi-user application, consider thread safety, validation, persistence, and ID generation.

20. StudentService and Constructor Injection

```
@Service
public class StudentService {
    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public Student createStudent(Student student) {
        return repository.save(student);
    }

    public Student getStudent(Long id) {
        return repository.findById(id);
    }

    public List<Student> getAllStudents() {
        return repository.findAll();
    }
}
```

- The service delegates to the repository in this simple example.
- Constructor injection makes the dependency explicit.
- When a class has one constructor, Spring can inject its dependency without requiring `@Autowired` on that constructor.

21. StudentController and REST Endpoints

```
@RestController
@RequestMapping("/students")
public class StudentController {
    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<Student> create(
        @RequestBody Student student) {
        Student saved = service.createStudent(student);
        return ResponseEntity.ok(saved);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getOne(
        @PathVariable Long id) {
        Student student = service.getStudent(id);
        if (student == null) {
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(student);
    }

    @GetMapping
    public ResponseEntity<List<Student>> getAll() {
        return ResponseEntity.ok(service.getAllStudents());
    }
}
```

This is a teaching example reflecting the lecture's main endpoints. A production API would normally validate input, generate IDs safely, and return 201 Created for a successful creation.

22. Test the Student API

Create a student

POST http://localhost:8080/students
Content-Type: application/json

```
{  
  "id": 1,  
  "name": "Aditya",  
  "email": "aditya@example.com"  
}
```

Get one student

GET http://localhost:8080/students/1

Get all students

GET http://localhost:8080/students

Use Postman or another HTTP client. The endpoint and HTTP method together determine which controller method should be selected.

23. Manual Web Configuration

Without Spring Boot auto-configuration, the application must explicitly configure Spring's web infrastructure.

```
@Configuration
@ComponentScan("in.dotsquares")
@EnableWebMvc
public class WebConfig {
}
```

- `@Configuration` marks a configuration class.
- `@ComponentScan` tells Spring where to find controllers, services, and repositories.
- `@EnableWebMvc` enables Spring MVC configuration in the manual setup shown in the lecture.
- Additional beans or configuration may be required for the chosen server setup and for views/static resources.

24. What the Main Method Must Configure

The lecture manually creates embedded Tomcat, sets the port, prepares a web application context, registers the `DispatcherServlet`, and starts the server. This is infrastructure or boilerplate code—not business logic.

Main method
1. Create Tomcat
2. Set port (e.g. 8080)
3. Configure web context
4. Register <code>DispatcherServlet</code>
5. Connect Spring application context
6. Start Tomcat

The exact APIs and setup details depend on the versions of Tomcat and Spring MVC. Use the lecture's code as a version-specific example, not as universal copy-paste code.

25. Why Spring Boot Reduces Boilerplate

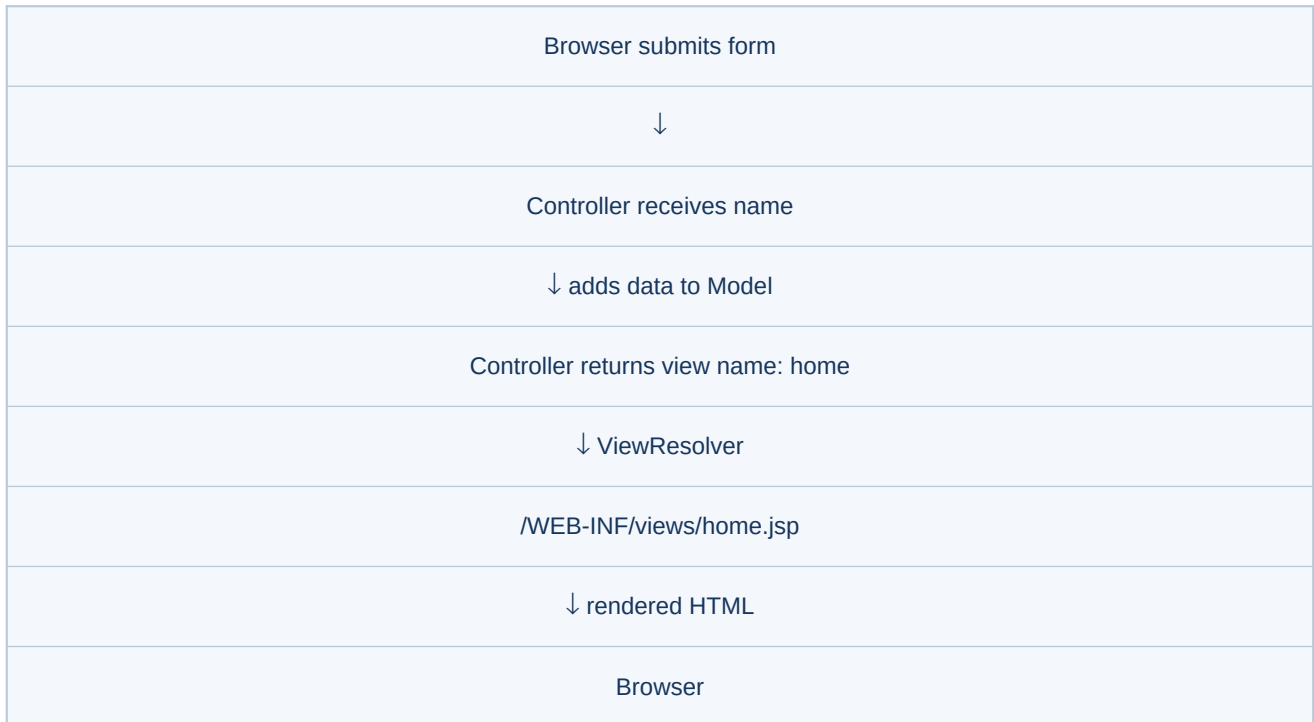
Manual Spring MVC
Dependencies + Tomcat setup
Servlet registration + context
MVC / component configuration
More setup before business logic

Spring Boot
Starter dependencies
Auto-configuration
Embedded server defaults
Start writing application logic sooner

The key idea is not that Spring Boot removes the MVC request flow. It automates much of the configuration around that flow.

26. JSP: Returning a Web Page Instead of JSON

Spring MVC can also serve traditional server-rendered web pages. The lecture demonstrates JSP (JavaServer Pages) as a basic view technology. JSP is legacy in many modern projects, but it helps illustrate the View part of MVC.



- REST endpoint: commonly returns data such as JSON.
- MVC view endpoint: returns a view name that is resolved to a page.
- A Model carries data from the controller to the view.

27. ViewResolver, JSP and WEB-INF/views

A ViewResolver maps the logical view name returned by a controller to the actual view file.

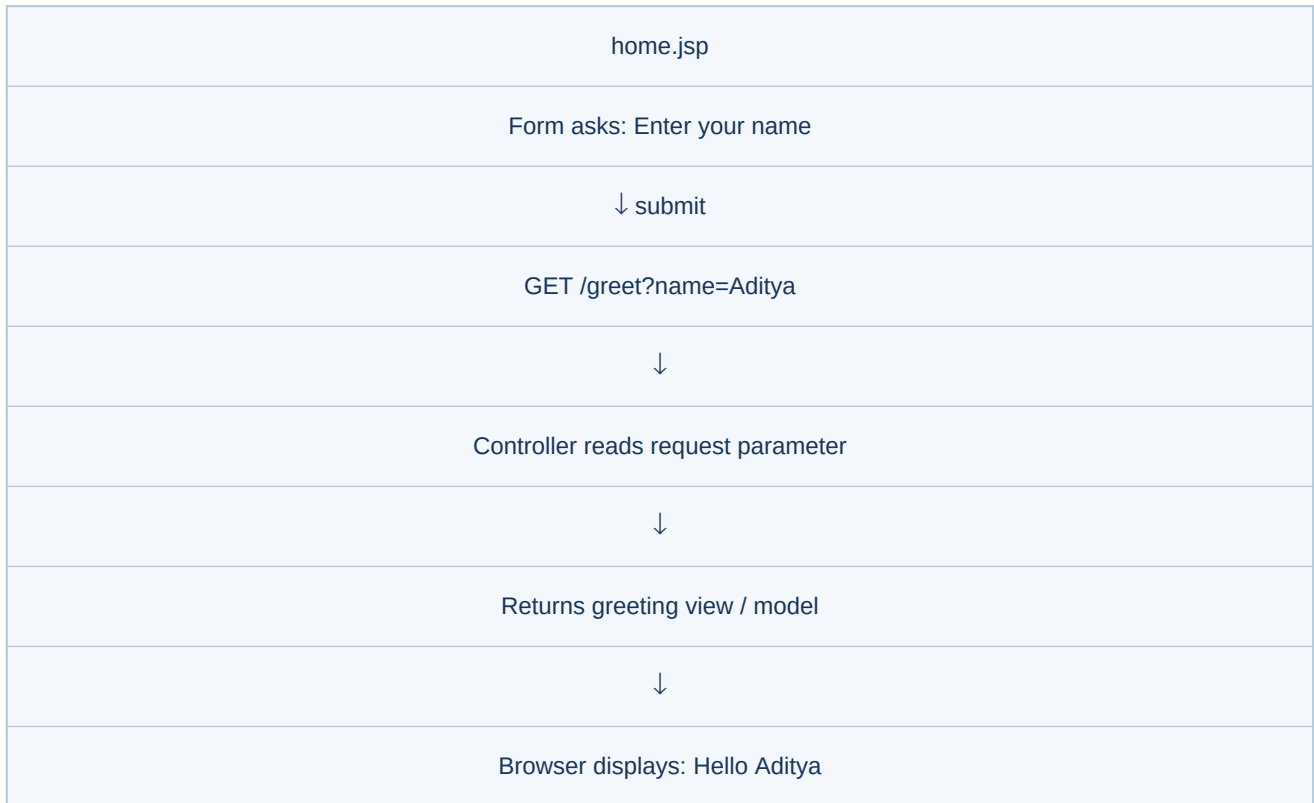
```
// Controller concept
@GetMapping("/")
public String home(Model model) {
    model.addAttribute("name", "Aditya");
    return "home";
}

// Example view resolver settings
prefix = "/WEB-INF/views/"
suffix = ".jsp"

// "home" resolves to:
// /WEB-INF/views/home.jsp
```

- The prefix identifies the view directory.
- The suffix identifies the file extension.
- The returned string is a logical view name, not necessarily a URL.
- The view renders the data supplied by the controller's model.

28. JSP Form Example — Hello Name



```
// Simplified controller idea
@GetMapping("/greet")
public String greet(
    @RequestParam String name,
    Model model) {
    model.addAttribute("name", name);
    return "greeting";
}
```

The form field is submitted as a request parameter. The controller reads it and adds the value to the model for the view to display.

29. Static CSS in Manual MVC

The lecture notes that static CSS can require extra MVC configuration in a manual setup. If static resource handling is not configured correctly, the JSP may load while its stylesheet does not.

- Place static files in a location supported by the app's configuration.
- Configure resource handling when needed.
- Check browser developer tools and the network response if a CSS file fails to load.
- The exact configuration depends on how the web application is assembled and deployed.

30. Final Architecture — One Complete Picture



This is the core mental model of the lecture. The real framework contains more internal components, but these are the major pieces to understand first.

31. Interview Revision

What is Spring MVC?

Spring Framework's web framework for REST APIs and web applications.

What is DispatcherServlet?

The central servlet / front controller that receives requests and coordinates request handling.

Does the controller listen on port 8080?

No. Tomcat listens for HTTP requests; Spring MVC routes requests to controller methods.

What does HandlerMapping do?

Finds the handler that matches the request, including its path and HTTP method.

What is the role of the IoC container?

Creates and manages Spring beans such as controllers, services, and repositories.

How does @RequestBody work?

Spring reads the request body and uses an HTTP message converter, often backed by Jackson, to bind JSON to a Java object.

What is the difference between @PathVariable and @RequestParam?

Path variables are part of the URL path; request parameters are key-value values in the query string or request parameters.

Why use a HashMap in the demo?

To mimic data storage without introducing a database or JPA into an architecture lesson.

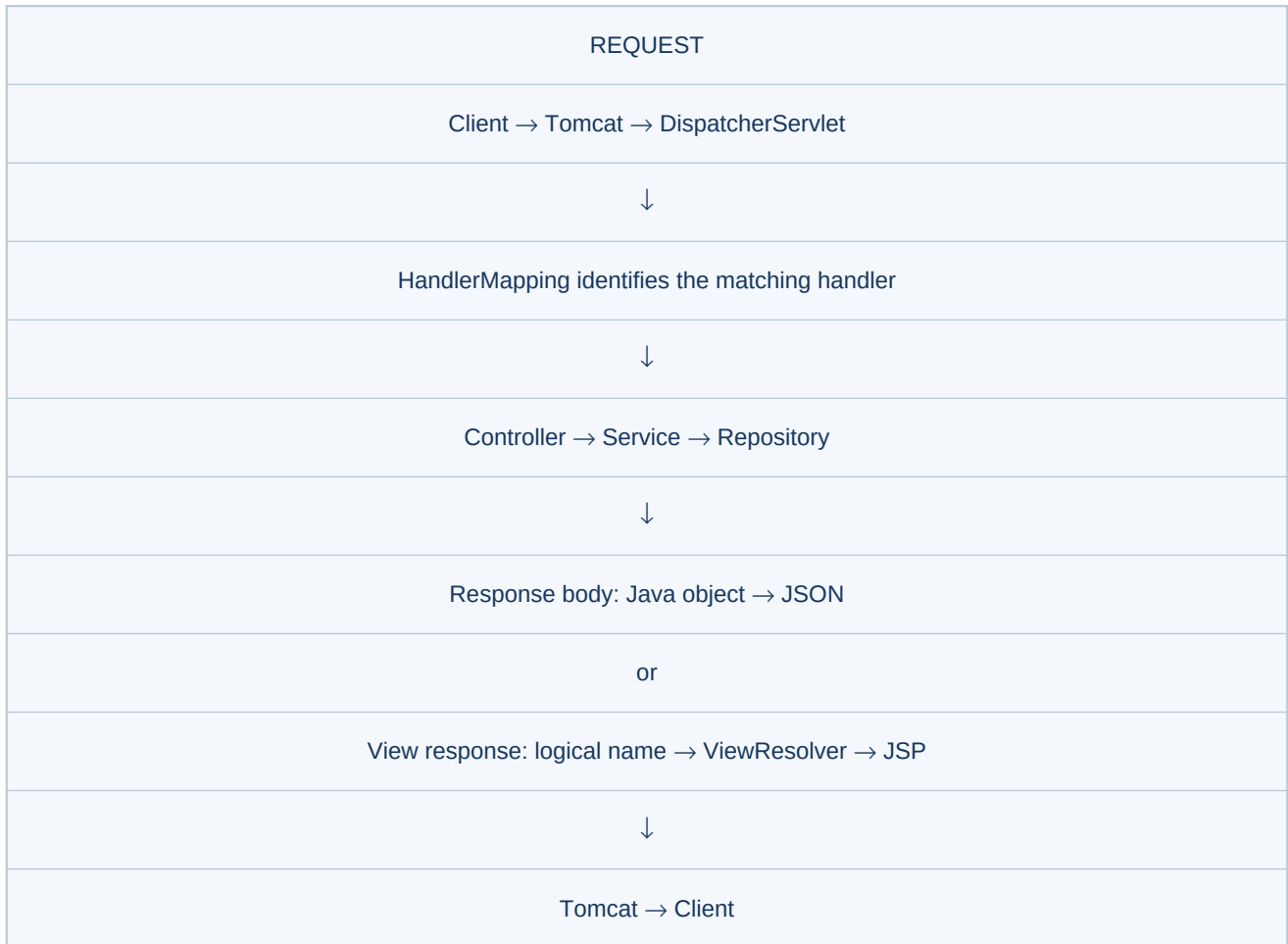
What does ViewResolver do?

Maps a logical view name to a concrete view, such as a JSP file.

Why is Spring Boot easier?

It auto-configures much of the common setup so developers write less boilerplate.

32. One-Page Cheat Sheet



- **Spring MVC:** web framework.
- **Spring Boot:** simplifies setup and auto-configuration.
- **Tomcat:** servlet container / HTTP server.
- **DispatcherServlet:** central request entry point.
- **HandlerMapping:** finds the matching controller method.
- **IoC container:** manages Spring beans.
- **@RequestBody:** body to Java object.
- **@PathVariable:** value from URL path.
- **@RequestParam:** query/request parameter.
- **ViewResolver:** logical view name to JSP/view.

Study tip: first memorize the request-flow diagram. Then explain each component in one sentence. Finally, practise tracing POST /students and GET /students/1 from client to response.